Compiler Project - **OhNoCamel**

DEVELOPER DOCUMENTATION

Team

1. GOMEZ ROSALES Laura Sofia
2. LUGO NOMESQUE José Francisco
3. SHIRAKAWA Jin
4. MUTHAIYAH Shaiva
5. NASSIR Ayda
6. BALERAS Timothé

Team 1
INTRODUCTION 3
Project Overview and Objectives 3
Compiler Architecture 3
FRONT END 3
LEXING AND PARSING 4
Lexer 4
Parser 5
Id 6
Type Checking 6
K-Normalisation 8
Alpha Conversion 9
Nested Let Reduction 10
Closure Conversion 11
ASML Generation 13
BACK END 15
Optimization local 15
Register Allocation 16
ARM Generation 18
TESTING 21
Invalid Type Checking 21
Optimization 25
Local Optimization 25
GLOSSARY 26
Technical terms and definitions 26
References 27

INTRODUCTION

This compilation project aims to design and implement a functional compiler for the subset of the OCaml language, known as MinCaml , following a structure and increment methodology. The project involves collaborative development using git lab and emphasizes a clear separation between front end and backend connected in integration in the middle through the intermediate language ASML. By progressively transforming MinCaml programs into ARM assembly code, the project provides practical insight into real world compiler construction.

Overall, this project serves as a comprehensive introduction to compilation techniques , toolchains and team based software engineering developments.

Project Overview and Objectives

The goal of this project is to create a working compiler that reliably handles a selected subset of the MinCaml language, which tasks are defined in given html document, therefore rather than covering all language features , the focus is on implementing the requested tasks namely simple arithmetic expression ,call to external functions,functions (let rec),if-then-else,closures,tuple, arrays from front end to backend pipeline . Thus the project aims to build a clear pipeline from the source code to ARM assembly using ASML as intermediate steps. Equal importance is given to code organization , reliability and teamwork throughout development. Testing is used continuously to ensure that each component behaves as expected.

Compiler Architecture

A compiler is software that translates source code written by a programmer into machine code that the computer understands. While doing this, it also checks for errors and improves the efficiency of the program.

FRONT END

The Frontend is first stage in the compiler in the compilation stage. It reads the source code and checks whether it's written correctly or not.

LEXING AND PARSING

Lexer

The lexer is the first stage in compilation, in which the lexer transforms the streams of units in to tokens such as integers, keywords, identifiers and operators. The second importance of the lexer is simplification by removing whitespace and comments. The third most important use of lexers is enforcing language rules as it enforces the recognition of keywords such as if, let, rec and helps the rest of the stages not to be confused with the identifier.

The section then focused on explaining major functions which related to our task in the project.

Function 1 Integer Values

An integer is defined as a sequence of one or more numeric digits. The function then describes how these integer literals are recognized by the lexer. When the lexer encounters such integer sequences it then converts it to integer tokens that represent its numeric values .

Function 2 SImple arithmetic expression

An arithmetic expression can be defined as the mathematical operation between two or more numbers to produce a result .For this project our focus of operands are integer numbers as float numbers were given as optional tasks. During the simple arithmetic operation the lexer is responsible for reading each character, grouping them and converting tokens of their type then passing the output for the parser for further process in compilation. Some of the functionalities in the tokenization are discussed below.

```
| digit+
   { INT(int_of_string (Lexing.lexeme lexbuf)) } responsible for tokenizing the integer numbers
| '-'
   { MINUS }
```

Responsible for grouping and character to MINUS token etc for the rest of the arithmetic operations.

Function 3 Array creation

The keyword Array.create is recognized as a single token by the lexer and its used to represent the creation of new array structure in the language. For example if we input Array.create 5 the lexer then outputs array creation token followed by an integer token representing the array size.

Function 4 Condition Statements (if then else)

This function describes the condition control flow. The keywords such as if, then and else are treated as reserved words and will be converted into tokens. These tokens are then used by the parser to build conditional expressions.

Function 5 Local Bindings (let-in)

This function explains how local variable bindings are recognized. The key word let introduces new binding and it specifies the scope in which the binding is valid. These keywords are then tokenized separately to support the scoped expressions. The output of the lexer is then the tokens separately that support scoped expressions.

Function 6 Recursive Definition (let rec)

This function describes recursive bindings. The keyword rec is used together with let to allow functions or values to refer to itself. Rec is then recognized by the lexer as a reserved keyword. The output of such a function for the let rec is then tokens representing a recursive definition.

Parser

The parser is responsible for taking the tokens from the lexer and checking whether they follow the language grammar or not. It organizes the tokens into a structural syntax tree (AST) that represents the program's meaning. For the following functions we will then continue the next stage from the lexer and explain the basic functions implemented in the project .

Simple arithmetic expression

| exp PLUS exp { Add($1,$3)} handles addition

| exp MINUS exp { Add($1,$3)} handles subtraction

If else

| IF exp THEN exp ELSE exp { If($2, $4, $6) }

Builds an IF node in the AST with condition, then branch and else branch

External/function calls

| exp actual_args { App($1, $2) }

These rules tell the parser how to recognize arithmetic expressions and convert them in to the corresponding syntax AST nodes to be processed by the compiler.

Id

This module provides functionality to create unique variable identifiers and temporary names, useful for compiler intermediate representations.

All identifier generation functions use a shared global counter to ensure uniqueness across all generated identifiers within the program's execution. The module also provides a {!type:M} map structure for storing identifiers indexed by strings.

Key Functions:

- **[to_string]** returns the string representation of identifier [x]
- **[genid]** is used by the parser, it generates a unique identifier of the form "?v<n>" where [<n>] is an incrementing counter. Used when the user writes code that implies a variable but doesn't name it
    - _ (Wildcard). let _ = ...
    - print_int 1; print_int 2 ➔ let ?v1 = print_int 1 in print_int 2
- **[genid_ s]** generates a unique identifier of the form "<s>.<n>" where [<s>] is a provided prefix string and [<n>] is an incrementing counter. Useful for creating scoped identifiers with a specific namespace. Also, this structure was chosen to differentiate variables called [v<n>] from variables created/renamed in k normalization and alpha conversion process.
- **[gentmp]** generates a temporary identifier of the form "v<n>" where [<n>] is an incrementing counter. The type parameter [typ] allows context-aware temporary generation, though it is currently unused.
    - 1+2 ➔ let v2 = 1 in let v3 = 2 in v2 + v3

Type Checking

This module implements a type checker with type inference capabilities for a simplified functional language. It supports:

- **Primitive types:** Unit, Bool, Int
- **Composite types:** Functions, Tuples, and Arrays
- **Type variables** with unification-based inference
- **Let-bindings**, recursive function definitions, and external function detection

Key Components:

- **[deref_typ]:** Dereferences type variables by following pointer chains to their concrete types. Path compression is applied to optimize future lookups.

- **[default_type_to_int]:** A post-inference pass that resolves any remaining uninstantiated type variables to Int. This ensures the backend receives concrete types for all variables.
- **[unify]:** Implements the Robinson unification algorithm to ensure two types are compatible. Handles:
    - Primitive type matching (Unit, Bool, Int, Float)
    - Function type unification to verify arity (argument count) and recursively unifies parameter/return types
    - Checks that tuple lengths match and recursively unifies each element
    - Recursively unifies the content type of the arrays
    - Type variable binding to perform safe assignment of types to type variables
- **[infer]:** The main recursive function for bidirectional type checking. It carries an environment mapping variables to types.

For each expression:

- As Math requires Int and Logic requires Bool, for unary and binary operations mean that primitives return their literal types after verify operand compatibility
- Conditionals as **If** expressions ensure the condition is Bool and unifies the types of the **then** and **else** branches
- Let-bindings add typed variables to the environment
- Variable lookup searches local and external (global) environments
- Function definitions (LetRec) establish recursive function types
- Function applications verify argument counts and types
- **For Tuples:**
    - **Creation:** Infers types of all elements to construct a Tuple type
    - **Destruction (LetTuple):** Unifies the bound expression with a Tuple structure and adds the extracted variables to the environment
- **For Arrays:**
    - **Creation:** Enforces that the size is Int and infers the array content type
    - **Access/Modification:** Enforces that the target is an Array and the index is an Int
- **[f]:** The entry point that:
- Resets the external function registry
- Runs type inference on the entire AST
- Attempts unification with Unit (main entry point type)
- Resolves all type variables to concrete types applying default_to_int
- Returns the original AST as types are updated in-place via references

Exception: Type_Error is raised on type mismatches with diagnostic information (e.g., adding a Bool to an Int, or mismatching Tuple lengths)

Limitations: Float operations (FAdd, FSub, FMul, FDiv) and Float Arrays are explicitly unsupported and will trigger a runtime failure if encountered

K-Normalisation

This module implements k-normalization, a transformation that converts arbitrary nested expressions from the source AST into a simpler intermediate form where complex expressions are decomposed into a sequence of let-bindings, ensuring that:

- Function arguments are always simple variables or constants
- Intermediate calculations are bound to temporary variables
- Control flow is simplified to explicit if-then-else constructs

Types:

- **[t]**: The k-normal form AST with constructors for:
    - Unit, Int (Booleans are converted to Ints 1/0)
    - Not, Neg, Add, Sub, Mul and Div, operands are variables, not expressions
    - IfEq, IfLE: Conditional branches based on comparisons
    - Let (variable binding), LetRec (function definition)
    - Var, App: Variable references and function applications
    - Tuple, LetTuple: Tuple construction and destructuring
    - Get, Put: Array access and update
    - ExtArray, ExtFunApp: External array and function support
- **[fundef]**: Function definition with name, arguments, and body

Key functions:

- **[type_of]:** A helper function that infers the type of a K-Normal expression by inspecting its constructor (e.g., Add implies Int). This allows correct type annotation for generated temporary variables.
- **[insert_let]:** Helper that wraps expressions in let-bindings when needed. It takes an expression e and a continuation k.
    - If **e** is already a "simple" value (like a variable), it passes it directly to **k**
    - If **e** is a complex computation, it generates a temporary variable **t**, creates a binding **let t = e**, and passes **t** to **k**. This ensures nested expressions like **(1+2) + 3** are flattened into **let t1=1+2 in t1+3**
- **[f]:** Main transformation function converting [Syntax.t] to [KNormal.t] form
    - Converts boolean values true/false into integers 1/0
    - Optimizes Not by swapping if-branches

    - Simplify implicit boolean checks (e.g., if x then ...) into explicit comparisons against zero (if x != 0)
    - Recursively normalizes the function and all arguments. It checks the function name against a known list of external symbols (e.g., print_int, create_array) to distinguish between closure calls **(App)** and library calls **(ExtFunApp)**
    - For **Tuples,** it flattens argument expressions before tuple creation
    - Converts **Array.create** syntax into an external call to **create_array**
- **[to_string]:** Pretty-prints k-normal expressions for debugging

Limitations:

- There is a limit list for External Function Call. This list can be extended according to the compiler's needs to recognize library calls.
    - create_array
    - create_float_array
    - print_int
    - print_newline
    - sin
    - cos
    - sqrt
    - abs_float
    - int_of_float
    - float_of_int
    - truncate
- Float operations and standalone comparisons (Eq/LE outside if) are unsupported and will raise failure**.**

Alpha Conversion

This module performs alpha conversion on K-normalized expressions to rename variables into unique identifiers while preserving the semantics of the program. This step eliminates variable shadowing and ensures that subsequent compiler phases (like Register Allocation) can treat every variable name as a unique entity.

Key Components:

- **[find]**: A lookup utility. It searches the current environment map for a variable's renamed alias. If not found (e.g., for external variables like print_int), it returns the original name unchanged
- **[is_temp_var]:** Predicate to check if a variable is already a temporary/unique variable. A variable is considered temporary if it matches the pattern "v<digits>" (e.g., "v0", "v123"). Temporary variables are preserved as-is to avoid unnecessary renaming of already-unique identifiers.
- **[alpha_rec]:** The core recursive function that traverses the AST carrying an environment map (string -> string).
- For operations like Add(x, y) or App(f, args), it simply looks up all variables in the environment to replace them with their unique aliases
- **Variable Definition (Let):**
    1. Generates a fresh unique name for the bound variable (e.g., x becomes x.42)
    2. Adds the mapping x -> x.42 to the environment
    3. Recursively processes the body using the new environment
- **Function Definition (LetRec):**
    - Renames the function name itself
    - Renames all function arguments.
    - Processes the function body with an environment containing both the renamed arguments and the renamed function name (to support recursion)
- **Tuple Destructuring (LetTuple):**
    - Iterates through the list of variables bound by the tuples
    - Generates a name for each one and updates the environment before processing the body
- **[alpha]:** Entry point for alpha conversion. Initiates the recursive conversion process with an empty environment mapping.

Nested Let Reduction

This module provides types and functions for converting K-normalized AST expressions after alpha conversion, into simplified intermediate representation where let bindings are flattened and optimized.

For example: This nested let-binding expression (let x = (let y = ... in ...) in ...) is flattened into a linear sequence (let y = ... in let x = ... in ...). This reduction simplifies subsequent phases like Closure Conversion and Code Generation by reducing the recursion depth and making the data flow linear.

This pass guarantees that the right-hand side of a Let binding is never another Let binding (except within function bodies or conditional branches), producing a "staircase" of definitions that maps cleanly to a sequence of assembly instructions.

Types:

- **[ident]**: A unifying type representing an operand that can be either an **Integer Constant** (INT) or a **Variable Reference** (IDENT). This optimization allows operations like Add(x, 5) instead of forcing let t = 5 in Add(x, t)
- **[t]**: The Net-Let AST. Key difference from K-Normal form is that binary operations (Add, Sub, Mul, Div) and conditional checks (IfEq, IfLE) now accept ident as the second operand, supporting immediate integers.

Key Functions:

- **[let_build]:** The core flattening logic. It implements the associative law for let-bindings:

  let x = (let y = M in N) in K ➔ let y = M in (let x = N in K)

- **[net_let_rec]:** The recursive transformation function.
  - For primitives converts K-Normal variable references (Id.t) into the more flexible ident type where applicable (e.g., for math operations)
  - When encountering Let(label, t1, t2), it recursively processes t1. If t1 itself turns out to be a Let expression, let_build is invoked to flatten the hierarchy
  - For constructs that define scope boundaries (like LetRec bodies or If branches), it recurses into the sub-expressions without attempting to flatten across the boundary, this ensures the structure preservation
- **[net_let]:** The main entry point for the transformation pass

Closure Conversion

This module transforms the Net-Let intermediate representation into a lower-level form where all functions are lifted to the top level. This step bridges the gap between high-level nested functions and assembly labels. It explicitly constructs closure records for functions that capture free variables and distinguishes between direct function calls and closure invocations.

After this pass, the program consists only of global functions and explicit memory operations. A function like **let x = 1 in let f y = x + y** is transformed into a global function **f_code(y, env)** that loads **x** from **env** (using the offset tracked in **Env_closure**), and the main body allocates a closure **{ptr=f_code, x=1}**

Types:

- **[ident]:** Represents identifiers as either integer constants or variable IDs.
- **[t]:** Expression type in closure representation with variants for primitives, operations, control flow, function calls, and memory operations
    - **AppClo:** Invoking a function pointer retrieved from a closure{
    - **App:** Direct invocation of a known top-level label.
    - **New:** Allocating memory (used for closures).
    - **Get/Put:** Explicit memory access (used to load free variables from the closure environment)
- **[fundef]:** Represents a function definition (name, arguments, body)
- **[toplevel]:** Represents the program structure as a flat list of function definitions (Fun) followed by the main expression (Main). This eliminates nested LetRec nodes

Implementation Strategy - Two Pass Algorithm

The conversion is performed in two distinct passes to handle recursive dependencies and environment calculation:

1. **Analysis & Lifting (closure_fun_rec):**
    - Traverses the AST to identify nested functions (LetRec)
    - Lifts these functions to the top level global list
    - Populates the **Env_closure** structure by mapping every variable to its defining scope (function name). This allows the compiler to identify "free variables" (variables used in a function but defined outside)
    - Replaces the original function definition with a placeholder structure.
2. **Code Injection (apply_closure):**
    - Rewrites the function bodies and main expression based on the data collected in the first pass
    - Closure Allocation: Inserts New instructions to allocate closure records on the heap for functions that escape their scope
    - **Variable Loading:** Uses add_let_closure to inject Let bindings at the start of functions, loading captured free variables from the **%self** closure pointer into local registers using the offsets calculated by **Env_closure**

<u>Key functions:</u>

- [**Env_closure**] **(Helper Module)**:
    - Manages the complex environment tracking
    - **Data Structures:** Maps variable names to their definition scopes (closure) and tracks heap offsets (env) for captured variables within closure records

    - **need_closure_bool:** Determines if a variable usage requires capture by comparing the current function scope with the variable's definition scope.
    - **add_var_env:** Assigns unique 4-byte offsets to free variables, building the layout of the closure record.
- [**closure_t_rec**]: The primary recursive transformation function for expressions. It handles variable tracking updates and delegates control flow constructs
- [**apply_closure_t**]:
    - Generates the actual closure handling code
    - **initialise_closure_smart**: Allocates the closure record. It calculates the size (4 bytes for code pointer + 4 bytes per free variable) and emits Put instructions to store the function label and captured values.
    - **Call Optimization**: Checks a hash table of known functions to decide between generating App (direct jump) or AppClo (dynamic call)
- [**closure**]: The main entry point. It initializes the environment maps and orchestrates the two-pass transformation, returning a toplevel structure ready for ASML generation

ASML Generation

This module converts from the Closure intermediate representation to the ASML (Abstract Simple Machine Language) representation. This step lowers high-level constructs into a format that directly maps to assembly instructions, specifically handling explicit memory management for data structures.

Part of its key responsibilities is to convert high-level functions constructs (like tuples, Let-bindings) into explicit memory allocation, loads and stores. Also, it restructures the program from a nested expression tree into a flat list of top-level function definitions followed by main entry point [asml_generator]. Finally, it handles the layout of data structures in memory, i.e., 4-byte alignment.

Key Functions:

- **[convert_ident]:** A utility that translates operands from the Closure representation into ASML's ident_or_imm type, allowing operations to work seamlessly with both variable names and immediate integers.
- **[remove_type]:** Extracts the name from a (name, type) tuple pair in creation_function
- **[mem_store]:**
    - Extracts the name from a (name, type) tuple pair
    - Takes a pointer to allocated heap memory and a list of variables
    - Recursively generates a sequence of MemStore instructions, writing each variable into sequential 4-byte offsets ([ptr+0], [ptr+4], etc.)

    - Chains these stores together, ensuring the tuple is fully initialized before the continuation **rest** executes
- [**mem_load**]:
    - Implements the logic for Tuple Destructuring (LetTuple)
    - Takes a source pointer and a list of variables to bind
    - Recursively generates **MemLoad** instructions, reading from sequential 4-byte offsets and binding the values to the variables
- [**closure_rec**]:
    - Expression Translation
    - Converts atomic expressions (Integers, Variables) and arithmetic/logic operations directly to ASML equivalents
    - Translates memory access: **Get** becomes **MemLoad**, **Put** becomes **MemStore**
    - Translates control flow (**IfEq, IfLE**) by recursively processing the branches
    - Handles function calls:
        - **Direct Calls (App, ExtFunApp):** Translated to Call instructions, i.e., jump to label
        - **Closure Calls (AppClo):** Translated to CallClo, signaling the backend to load the function pointer from the closure environment
- [**closure_rec_let**]:
    - Statement Translation
    - **Tuple Creation:** Intercepts Let(x, Tuple(...), ...) patterns. It calculates the required size in bytes, generates a **New** instruction (malloc), and then calls **mem_store** to populate the memory
    - **Tuple Destruction:** Intercepts **LetTuple** patterns and invokes **mem_load** to extract fields
    - **Tuple as a return value:** Similar to the tuple creation logic, plus creating a new pointer to allocate the tuples variables. Then this pointer is the one who is returned.
    - **Standard Binding:** Converts standard **Let** bindings recursively
- [**creation_function**]:
    - Iterates through the list of function definitions in the Closure
    - Renames functions (prefixing with _ to avoid assembly keyword clashes)
    - Strips type information from arguments (as ASML is untyped)
    - Converts the function body using **closure_rec_let**
    - Separates the "Main" expression from the function definitions, producing the final ASML program structure
- [**asml_generator**]: Top-level entry point that converts a complete Closure program to ASML representation

BACK END

The BackEnd is the final stage of the compiler. It takes the intermediate code produced by the frontend and converts it into machine code.

Optimization local

This module does a local optimization of the code asml.

It takes an asml type and produces an optimized version of the asml of the same format.

The file is composed of three part :

- The initialization
- The recurrency part
- The elimination of useless “let”

The initialization:

The initialization phase is just an analysis of the code.

We store the variables that are associated to a fixed value (let x = 5 in ...) for example.

Inside a hash table call “var_association”

And the dependency for each variable in the var_dependancy hash table.

The recurrent part:

For each iteration of the file we optimize by replacing the var by the value of those associated in the hashtable or calculating the different arithmetic expression (see Section Optimization below)

After that we reanalysis the code modify in order to recalculate the dependency

And add a new var that has an immediate value.

Remove of useless assignation:

The last part is to remove the useless assignation (var are not use)

So we redo each instruction in a body and remove each variable that is present in the association file and have no dependency.

Register Allocation

Allocation Strategies

Two strategies are implemented to allocate registers, which are part of the conversion from ASML to ARM code. They are:

- **Spill-All Strategy:** This is a less efficient method that was initially implemented to allocate registers. The stack is treated as the primary storage space and the registers and temporary spaces to load from and into during operations. Some of the important components and functions are.
    - **[stack_map] (Hashtbl):** A dictionary that maps variable names (id) to their memory location (offset) on the stack. This ensures that every time the compiler sees variable x, it refers to the same spot in RAM.
    - **[stack_next] (Ref):** A pointer tracking the next available memory address on the stack. It starts at -12 (leaving room for the saved Frame Pointer and Link Register) and moves down by 4 bytes for every new variable.
    - **[counter] (Ref):** A global integer used to create unique labels for assembly branches (e.g., then_1, then_2). This prevents label collisions in nested if statements.

        Key Functions:

    - **[emit_exp] (Expression Emitter)**: Translates high-level operations (like addition, division, or function calls) into specific ARM assembly sequences. It uses registers r0, r1, and r2 as temporary workspaces to perform calculations before passing the result upward.
    - **[emit_asmt] (Assignment Emitter):** Handles the logic of LET bindings by first calling emit_exp and then immediately storing the result from r0 into a stack slot. It recursively processes the "next" instruction in the code, ensuring the entire program is traversed line by line.
    - **[generate_assembly] (Program Orchestrator):** Handles the high-level structure of the assembly file, including function headers, stack setup (prologue), and cleanup (epilogue). It manages the transition between different functions and ensures the Main entry point is correctly formatted for the OS.
- **Linear Greedy Allocation:** This is a more efficient strategy that uses registers to store values based on the provided conventions and uses the stack when no more registers are available. This strategy is divided into two parts, Part 1 and Part 2, which perform register allocation in Part 1 and ARM translation in Part 2 based on what is defined in the previous part. Some of the important components and functions are:

- **[allocation_map] (Hashtbl):** It function likes an address book. It stores the final location of every variable, mapping an Id to either a Reg (physical register) or Stack (memory offset).
- **[all_regs] (String List):** It defines the pool of available callee-saved registers (r4–r10) that the allocator is allowed to give out to local variables.
- **[stack_count] (Ref):** This is an overflow counter. It tracks how many variables failed to get a register and were "spilled" to the stack, allowing the compiler to calculate the total frame size.

Key Functions (Part 1: Allocation Strategy):

- **[allocate_registers asmt args]:** The entry point for the allocation process. It first resets the state, then maps the function parameters (args) to the standard ARM argument registers (r0–r3). If a function has more than 4 arguments, it calculates a stack offset for them. Once arguments are handled, it uses the scan function to process the function body.
- **[scan] (Recursive Function):** A recursive visitor that traverses the program. It looks for LET bindings (where variables are created). If it finds a new variable, it tries to pop a register from free_regs.
- **[get_loc id]:** A simple lookup tool used later by the code generator. It queries the allocation_map. If the variable isn't found, it defaults to r0.
- **[get_stack_size]:** Determines how much memory to reserve in the function prologue. It takes the number of spilled variables, multiplies by 4 (bytes), and applies an 8-byte alignment mask (land (lnot 7)).
- **[reset_allocation]:** Clears the slate for a new function. It empties the hash table and sets the stack counter back to zero.

Key Functions (Part 2: ARM Generation):

- **[load_val]:** Moves a value into a hardware register. It checks if a variable is already in a register. If it is, it performs a fast register-to-register mov; if it's on the stack, it performs a memory ldr.
- **[gen_label]:** Creates unique, incrementing string labels (like div_loop_0, div_loop_1) for assembly branching.
- **[emit_asmt] (Statement Emitter):** Processes LET bindings. It asks Part_1 where a variable should live; if it's a register, it tells emit_exp to calculate the value directly into that register.
- **[generate_assembly] (Function Orchestrator):** Manages the function's Prologue and Epilogue, including saving the "Callee-Saved" registers (r4-r10). It sets up the Frame Pointer (r11) with a specific offset (#32) to account for the 8 registers pushed onto the stack, ensuring memory addresses remain consistent.

These two strategies are implemented and do not overlap in functionality. They can be accessed using flags during execution.

ARM Generation

Arithmetic Operations (addition, subtraction, multiplication)

Addition, subtraction and multiplication are handled in the same way. For the first operand, if the get_loc function returns a register, the returned register that holds the operand value is used for the operation. If the function returns an offset of the stack, the value stored in the returned address is loaded to r2 and used for the operation. Similarly, for the second operand, unless the get_loc function returns a register, the operand value is stored in r3 and used for the operation. Using these, ARM assembly instructions such as examples below are generated:

add r0, r0, r1

Operation code | destination register | first operand | second operand

Arithmetic Operation (division)

Our approach is to create a loop which subtracts the divisor from the dividend and use the difference as a new dividend for the next loop. The loop is executed until the divisor becomes greater than the dividend (the comparison is performed by the **cmp** instruction) and the result is incremented by 1 in each loop. The dividend is set in r2 and the divisor is set in r3. For example, the division of 12 / 5 is performed as below:

Dividend	Devisor	Difference	Result	Step
12	5	7	1	12>5, performs subtraction, incrementing the result and repeats the loop
7	5	2	2	7>5, performs subtraction, incrementing the result and repeats the loop
2	5	-	-	2>5, not true and jumps to the finish branch

Table 1.1 Arithmetic Operation

If-then-else

The If-then-else operation is performed in the emit_if function which receives the following arguments:

- Condition: branches “beq”, “ble” and “bge”
- l_operand: the first operand
- r_operand: the second operand
- inst_true: the set of instruction when the condition is true
- Inst_false: the set of instructions when the condition is false

Since the if-then-else operation requires jumping to different blocks of instructions depending on whether the condition is true or false, labels “then” and “else_end” are used. For nested conditions, unique labels are generated by the gen_label function that each label has a different number in its suffix in order for the compiler to distinguish which block of instructions it needs to process. The same way as the arithmetic operations, the first operand is firstly moved/loaded to r2 and the second operand is moved/loaded to r3 unless the get_loc function finds it in other specific registers. Then, these operands are compared by the **cmp** instruction. If the condition is false, asmt2 is performed in the then block and jumps to the end block by the **b** instruction after completion. Otherwise, asmt1 is performed in the then block and jumps to the end block.

Key Functions:

- **[get_loc]:**
  - Returns the location where a variable is allocated in the allocation map
  - If it is not present in the map, the function catches the Not_found exception and sets r0 as the default register
- **[gen_label]:**
  - Creates unique labels for a compiler to know to which branch it needs to jump, and in which branch it needs to process the code
  - By “unique” labels, it means that each label has a number in its suffix which the number is incremented every time the function is called and this prevents each label from being duplicated
- **[load_val]:**
  - Loads a value to a destination register
  - For a constant value and a register returned by the get_loc function, the value is set to the destination register by the **mov** instructions
  - For a value stored in the stack, the value is directly loaded from the stack using the offset
- **[emit_asmt]:**
  - Generates ARM assembly code for the following cases: LET and EXP (conditional expressions such as ifEQ, ifLE, ifGE and other expressions not applied to the ones mentioned)

Function Calls (Call)

Function calls follow the standard ARM Calling Convention for register usage and stack management. This component also handles recursive calls.

Arguments

- **[func_name]:** The identifier of the function to be called.
- **[args]**: A list of arguments to be passed to the function.Implementation

Details

- **Register Passing (r0-r3):** The first four arguments are loaded directly into registers r0 through r3.
- **Stack Spilling:** If there are more than 4 arguments, the remaining values are loaded into r12 (intra-procedure-call scratch register) and then stored on the stack (r13) with a calculated offset: $offset = (index - 4) \times 4$.
- **Execution:** The bl (Branch with Link) instruction is used to jump to the function while saving the return address.
- **Return Value:** The result is expected in r0. If the dest_reg is not r0, a mov instruction transfers the result to the correct location.

Array Management

The backend handles array operations by calculating memory offsets based on the word size (4 bytes) using logical shifts.

Implementation Details

- **A.MemLoad / A.MemStore:** Uses the ARM scaling instruction lsl #2 (Logical Shift Left by 2) to multiply the index by 4, converting the index into a byte offset.
- **A.New:** Calculates total byte size ($size \times 4$), calls the _min_caml_alloc external function, and moves the resulting pointer from r0 to the destination register.

Key Functions

- **[_min_caml_alloc]:** Allocates a block of memory on the heap by incrementing a global pointer (heap_ptr) by the requested number of bytes. The function saves the current pointer address to return to the caller and then updates the global memory with the new address for future allocations.
- **[_min_caml_create_array]:** Initializes a new array in memory given a size and an initial value. It converts the element count into bytes (multiplying by 4), calls _min_caml_alloc to

get memory, and then runs a loop to fill every index with the provided value before returning the array's address.

TESTING

The codes have been tested with the examples provided, and new ones have been created to evaluate valid and invalid situations. Here are some examples of output:

Invalid Type Checking

Strict Type error:

This expression has type int due to minus (-) operation, but an expression got type bool as second operand

Test Code	Output
1 - **true**	[Type Error] Mismatch between: bool and: int

Table 1.2 Strict Type error test-output

Branches with different type error:

Expression `if_then_else` expected the same type in both branches. In this case, the Type Checking doesn't know which branch is correct (`then` branch or `else` branch), it just found a mismatch between returning values.

Test Code	Output
if true then **1** else **false**	[Type Error] Mismatch between: int . and: bool

Table 1.3 Branches with different type test-output

Statement condition is not bool error:

This expression has type int, but an expression was expected of type bool because it is in the condition of an if-statement. In other words, while the branches have matching types, the condition is invalid.

Test Code	Output
if **1** then 10 else 20	[Type Error] Mismatch between: int and: bool

Table 1.4 statement condition test-output

Multiples errors:

This code has 2 errors:

1. **Unbound value [y]:** This error occurs when the variable y is used in the expression but never defined.
2. **Strict Type:** x is bound to be a boolean in if condition, but in then branch x is trying to add 1 to it (x + 1). The + operator expects integers.

Test Code	Output
let x = y in if x then x + 1 else 0	[Type Error] Mismatch between: bool and: int

Table 1.5 Multiple errors test-output

Argument mismatch error:

The function f is defined as `int -> int`, but it is called with true, which is a bool.

Test Code	Output
let rec f x = x + 1 in f true	[Type Error] Mismatch between: int and: bool

Table 1.6 argument mismatch test-output

Arity mismatch error:

- **(? -> ?):** Since x isn't used with math, its type is unknown (?). So f has type ? -> ? (Takes 1 argument, returns it).
- (int -> int -> ?): The compiler sees the call is passing 2 arguments. Therefore, the compiler expects f to be a function that takes an int (the 1), returns another function that takes an int (the 2), and returns something (?).

Test Code	Output
let rec f x = x in f 1 2	[Type Error] Mismatch between: (? -> ?) and: (int -> int -> ?)

Table 1.7 Arity mismatch test-output

Invalid type insertion into array error:

The compiler only supports integers array, due to this, trying to insert a boolean into it, it is a type error

Test Code	Output
let a = Array.create 10 0 in (* Array of Int *) a.(0) <- **true**	[Type Error] Mismatch between: int and: bool

Table 1.8 Invalid type insertion into array error test-output

Array Index Type Error (for INT)

Test Code	Output
let a = Array.create 10 0 in a.(**true**)	[Type Error] Mismatch between: bool and: int

Table 1.9 Array index type error test-output

Accessing a non-array error:

`x` is defined as an integer, but it is called in a context that expects an `'a array` (an array of any type [?])

Test Code	Output
let **x** = 10 in **x.(0)**	[Type Error] Mismatch between: int and: [?]

Table 2.0 Accessing non- array error test-output

Tuple size mismatch:

Expected 3 arguments of any type, but just found 2.

Test Code	Output
let t = **(1, 2)** in	[Type Error]

let **(x, y, z)** = t in x	Mismatch between: (? * ? * ?) and: (int * int)

Table 2.1 Tuple size mismatch error test-output

Invalid type inside tuple:

Trying to add `Int + Bool`

Test Code	Output
let t = (**1**, true) in let (x, y) = t in **x** + y	[Type Error] Mismatch between: bool and: int

Table 2.2 Invalid type inside tupleerror test-output

Optimization

Local Optimization

The compiler provides a small possible optimization.

It's a local optimization. Meaning the optimization will apply separately from the body of one function/or the main body of the script.

What optimization does ?

- **Useless assignments**

It eliminated useless assignments.

which is a let instruction where the variable used to store the result is never used.

Like for example, the following script :

Code before optimization	Code after optimization
Let x = 5 in Let y = 2 in Call _print_int x	Let x = 5 in Call _print_int

In the example just above

Y is defined but never used in the script and thus it will be eliminated.

Remark :

In mincaml there is no instruction ending with the character ';' unlike original Ocaml which gives the possibility to write multiple instructions without need to use a let.

To counter it we use multiple let where the variable is used for nothing and contained in our case unit. And thus we do not try to eliminate those "let" .

We only eliminate let that contain a int on the right side

- **Replace of defined variable**

  We know the value of some variable at compile time,so we can try to replace this variable on the body.

  Some variables cannot be determined at compile time. In our compiler those variables come from a closure or the parameter the function takes.

We try to replace this variable wherever we can while keeping respecting the Asml type for a body.

All the places where it is replaceable directly in the ASML type are all the places where the type `ident_or_imm` are present and which mean identification or immediate value.

- **Pre-calculation of simple arithmetic expression**

A lot of compilers actually evaluate some arithmetic expression during compile when they know both value and that can be determined.

For example:

Code before optimization	Code after optimization
Let x = 5 in Let y = 3 in Let ret = add 5 3 Call _print_int ret	Let ret = 8 in Call _print_int ret

If not all variables are determined at compilation then we cannot calculate the value with the operation for example only one is determined at compile time. We try to replace the second operand with the value and thus get arithmetic with a imédiate value.

Note that only add and mul are associative and thus we can try to swap both variables so we have an immediate value for the opération.this action is only performed with an optimization of level 2.

If it is level 1 he will not swap it.

And two registers would be available.

GLOSSARY

Technical terms and definitions

- **AST** - Abstract Syntax Tree, a structure representation of source code.
- **ASML** - An intermediate assembly language used between front end and back end.

References

- Hindley–Milner type system
- OCaml Programing - Type Checking
- The Compiler Frontend: Parsing and Type Checking
- A Crash Course for the MinCaml Compiler - Type Inference
- CS 6110 S17 Lecture 23 Type Inference and Unification
- File manipulation
- Hash Tables
- Branch and Call Sequences Explained